```
> # ============================================================
  # TauTimp Framework -- Teoria Onduscularа
  # Spherical Wave Equation with Variable Coefficient
  # H(r,theta,t) and Probability Density P(r,t)
  # Author: Viorel Popescu
  # Date  : April 2026
  # ============================================================

  restart;
  with(plots):
  with(plottools):

  # ------------------------------------------------------------
  # SECTION 1 -- Physical Constants and Cosmological Parameters
  # ------------------------------------------------------------

  printf("=== SECTION 1: Physical Constants ===\n");

  R_val  := 3.9228e22:          # cosmological radius [m]
  c_val  := 2.99792458e8:       # speed of light [m/s]
  Pi_val := evalf(Pi):

  printf("  R  = %.6e m\n", evalf(R_val));
  printf("  c  = %.6e m/s\n", evalf(c_val));

  # ------------------------------------------------------------
  # SECTION 2 -- The Wave Equation (symbolic display)
  # ------------------------------------------------------------

  printf("\n=== SECTION 2: TauTimp Wave Equation ===\n");

  # Symbolic PDE: (R/r)*Delta_f = (1/c^2)*d^2f/dt^2
  # Under spherical symmetry:
  # (R/r)*(diff(f,r,r) + 2/r*diff(f,r)) = (1/c^2)*diff(f,t,t)

  f := f(r, t):
  PDE_TauTimp := (R/r) * (diff(f, r, r) + (2/r)*diff(f, r))
               - (1/c^2)*diff(f, t, t) = 0;
  printf("  TauTimp PDE (spherical symmetry):\n");
  print(PDE_TauTimp);

  # ------------------------------------------------------------
  # SECTION 3 -- Separation of Variables: f(r,t) = g(r)*u(t)
  # ------------------------------------------------------------

  printf("\n=== SECTION 3: Separation of Variables ===\n");

  # Temporal ODE: u''(t) + lambda^2*c^2*u(t) = 0
  # Solution: u(t) = C1*cos(omega*t) + C2*sin(omega*t), omega = lambda*c
```

```
u_sol := C1*cos(omega*t) + C2*sin(omega*t);
printf("  Temporal solution u(t):\n");
print(u_sol);

# Radial ODE: g''(r) + (2/r)*g'(r) + (lambda^2*r/R)*g(r) = 0
g := g(r):
RadialODE := diff(g, r, r) + (2/r)*diff(g, r) + (lambda^2*r/R)*g = 0;
printf("  Radial ODE:\n");
print(RadialODE);

# ------------------------------------------------------------
# SECTION 4 -- Reduction to Airy Equation via g(r) = v(r)/r
# ------------------------------------------------------------

printf("\n=== SECTION 4: Airy Equation Reduction ===\n");

# Substitution g(r) = v(r)/r  =>  v''(zeta) - zeta*v = 0
# where  zeta = (lambda^2/R)^(1/3) * r

AiryEq := diff(v(zeta), zeta, zeta) - zeta*v(zeta) = 0;
printf("  Standard Airy equation:\n");
print(AiryEq);

# General Airy solution
v_general := D1*AiryAi(zeta) + D2*AiryBi(zeta);
printf("  General solution v(zeta):\n");
print(v_general);

# Regularity: keep only AiryAi (AiryBi diverges for large zeta)
printf("  Physical solution (D2=0, regularity at r->0):\n");
v_phys := D1*AiryAi(zeta);
print(v_phys);

# ------------------------------------------------------------
# SECTION 5 -- Eigenvalue Quantization from BC f(R,t)=0
# ------------------------------------------------------------

printf("\n=== SECTION 5: Eigenvalue Quantization ===\n");

# BC: g(R)=0  =>  AiryAi( (lambda^2/R)^(1/3)*R ) = 0
# i.e. zeta_n(R) = -|a_n|  where a_n are zeros of AiryAi

# Compute first 5 Airy zeros (negative)
N_modes := 5:
AiryZeros := Vector(N_modes):
for n from 1 to N_modes do
    # Initial guess from asymptotic formula: a_n ~ -(3*Pi*(4n-1)/8)^(2/3)
    guess_n := -(3*Pi*(4*n-1)/8)^(2/3):
    AiryZeros[n] := fsolve(AiryAi(x) = 0, x = evalf(guess_n) - 0.5
```

```
   .. evalf(guess_n) + 0.5);
end do:

printf("  Airy zeros a_n (zeros of Ai):\n");
for n from 1 to N_modes do
    printf("    a_%d = %.6f\n", n, evalf(AiryZeros[n]));
end do:

# Eigenvalues: lambda_n = |a_n|^(3/2) / R
# Eigenfrequencies: omega_n = c * |a_n|^(3/2) / R

lambda_vals := Vector(N_modes):
omega_vals  := Vector(N_modes):
for n from 1 to N_modes do
    lambda_vals[n] := abs(AiryZeros[n])^(3/2) / R_val;
    omega_vals[n]  := c_val * lambda_vals[n];
end do:

printf("\n  Eigenvalues lambda_n and omega_n:\n");
for n from 1 to N_modes do
    printf("    n=%d: lambda_%d = %.6e rad/m,  omega_%d = %.6e rad/s\n",
           n, n, evalf(lambda_vals[n]), n, evalf(omega_vals[n]));
end do:

# ------------------------------------------------------------
# SECTION 6 -- Radial Eigenfunctions g_n(r)
# ------------------------------------------------------------

printf("\n=== SECTION 6: Radial Eigenfunctions g_n(r) ===\n");

# g_n(r) = (1/r) * AiryAi( |a_n|*r/R )
# scaled to unit maximum for plotting

g_n := (n_idx, rr) -> AiryAi(abs(AiryZeros[n_idx]) * rr / R_val) / rr;


   # Normalise each mode for display using dimensionless variable s = r/R
   in [0,1]
# g_n_dim(s) = AiryAi( |a_n|*s ) / s    (s in [0,1])

g_dim := (n_idx, s) -> AiryAi(abs(evalf(AiryZeros[n_idx])) * s) / s;

printf("  Radial eigenfunction (dimensionless, s=r/R):\n");
printf("    g_n(s) = AiryAi(|a_n|*s) / s\n");

# ------------------------------------------------------------
# PLOT 1 -- Radial Eigenfunctions (first 4 modes)
# ------------------------------------------------------------

printf("\n=== PLOT 1: Radial Eigenfunctions g_n(s) ===\n");
```

```
colors_list := [blue, red, green, magenta]:
plots_g := []:
for n from 1 to 4 do
    local an_abs;
    an_abs := abs(evalf(AiryZeros[n]));
    # avoid singularity at s=0: start from small epsilon
    p_n := plot(
        AiryAi(an_abs * s) / s,
        s = 0.01 .. 1,
        color = colors_list[n],
        thickness = 2,
        legend = cat("n=", convert(n, string))
    ):
    plots_g := [op(plots_g), p_n];
end do:

plt1 := display(
    plots_g,
    title
   = "TauTimp: Radial Eigenfunctions g_n(s) = Ai(|a_n|*s)/s,  s = r/R",
    labels = ["s = r/R", "g_n(s)  [arb. units]"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes = BOXED,
    gridlines = true
):
printf("  Plot 1 generated.\n");

# ------------------------------------------------------------
# SECTION 7 -- Full Wave Field f(r,t) -- first mode, dimensionless
# ------------------------------------------------------------

printf("\n=== SECTION 7: Full Wave Field f(r,t) -- Mode 1 ===\n");

# f_1(s, tau) = AiryAi(|a_1|*s)/s * cos(omega_1 * tau)
# where tau = c*t/R  (dimensionless time)

a1_abs := abs(evalf(AiryZeros[1]));

f1_expr := (s, tau) -> AiryAi(a1_abs * s) / s * cos(a1_abs^(3/2) * tau);

printf("  f_1(s,tau) = Ai(|a1|*s)/s * cos(|a1|^(3/2)*tau)\n");
printf("  |a1| = %.6f\n", a1_abs);

# ------------------------------------------------------------
# PLOT 2 -- Spacetime surface f_1(s, tau)
# ------------------------------------------------------------

printf("\n=== PLOT 2: Spacetime Surface f_1(s,tau) ===\n");
```

```
plt2 := plot3d(
    AiryAi(a1_abs * s) / s * cos(a1_abs^(3/2) * tau),
    s   = 0.02 .. 1,
    tau = 0 .. 2*Pi,
    grid    = [50, 50],
    style   = surface,
    shading = zgrayscale,
    axes    = BOXED,
    orientation = [45, 70],
    title   = "TauTimp: f_1(s,tau) = Ai(|a1|*s)/s * cos(|a1|^(3/2)*tau)",
    labels  = ["s=r/R", "tau=ct/R", "f_1"],
    titlefont = [TIMES, BOLD, 12]
):
printf("  Plot 2 generated.\n");

# ------------------------------------------------------------
# SECTION 8 -- H(r,theta,t): angular reconstruction (ell=0)
# ------------------------------------------------------------

printf("\n=== SECTION 8: H(r,theta,t) -- Angular Field ===\n");

# For ell=0 (isotropic): H(r,theta,t) = f(r,t)
# For ell=1: H includes P1(cos theta) = cos(theta)
# Display H_1 = f_1(s,tau) * P1(cos(theta)) for illustration

# P1(cos theta) = cos(theta)
H1_expr := (s, th, tau) ->
    AiryAi(a1_abs * s) / s * cos(a1_abs^(3/2) * tau) * cos(th);

printf("  H_1(s,theta,tau) = f_1(s,tau)*cos(theta)  [ell=1 mode]\n");

# ------------------------------------------------------------
# PLOT 3 -- H_1 in polar cross-section at fixed tau=0
# ------------------------------------------------------------

printf("\n=== PLOT 3: Polar map of H_1(s,theta) at tau=0 ===\n");

# Convert to Cartesian: x = s*sin(theta), z = s*cos(theta)
# H_1 = Ai(|a1|*s)/s * cos(theta)

plt3 := plot3d(
    [s * sin(th), s * cos(th),
     AiryAi(a1_abs * s) / s * cos(th)],
    s  = 0.02 .. 1,
    th = 0 .. Pi,
    grid      = [40, 40],
    style     = surface,
    shading   = zgrayscale,
    axes      = BOXED,
    orientation = [30, 60],
```

```
    title
   = "TauTimp: H_1(r,theta,0) polar cross-section (ell=1, tau=0)",
    labels    = ["x=r*sin(theta)/R", "z=r*cos(theta)/R", "H_1"],
    titlefont = [TIMES, BOLD, 12]
):
printf("  Plot 3 generated.\n");

# ------------------------------------------------------------
# SECTION 9 -- Probability Density P(r,t): Wave PDE
# ------------------------------------------------------------

printf("\n=== SECTION 9: Probability Density PDE ===\n");

# PDE: d^2P/dt^2 - c^2 * d^2P/dr^2 = 0
# Boundary: P(R,0) = 0,  P(0,0) = 0,  int_0^R P dr = 1

P_func := P(r, t):
PDE_P := diff(P_func, t, t) - c^2 * diff(P_func, r, r) = 0;
printf("  Probability PDE:\n");
print(PDE_P);

# ------------------------------------------------------------
# SECTION 10 -- Degree-2 Polynomial: P(r,0) = A*r^2 + B*r + C
# ------------------------------------------------------------

printf("\n=== SECTION 10: Degree-2 Initial Profile P(r,0) ===\n");

# Three constraints:
#   (i)  P(0,0) = 0  =>  C = 0
#   (ii) P(R,0) = 0  =>  A*R^2 + B*R = 0  =>  B = -A*R
#   (iii)int_0^R P(r,0) dr = 1  =>  A*(-R^3/6) = 1  =>  A = -6/R^3

# Symbolic derivation
A_sym := -6/R^3;
B_sym :=  6/R^2;
C_sym := 0;

printf("  Symbolic coefficients:\n");
printf("    A = -6/R^3\n");
printf("    B =  6/R^2\n");
printf("    C =  0\n");

# Numerical values
A_num := evalf(-6 / R_val^3);
B_num := evalf( 6 / R_val^2);
C_num := 0.0;

printf("\n  Numerical values (R = %.4e m):\n", R_val);
printf("    A = %.6e m^-3\n", A_num);
printf("    B = %.6e m^-2\n", B_num);
```

```
printf("    C = 0\n");

# Verify constraints symbolically
P0_sym := A_sym * r^2 + B_sym * r;   # with C=0, R symbolic
printf("\n  P(r,0) symbolic form:\n");
print(P0_sym);

check_BC   := simplify(subs(r = R, P0_sym));
check_norm := simplify(int(P0_sym, r = 0..R));
printf("  Verification P(R,0) = %a  (should be 0)\n",  check_BC);
printf("  Verification integral = %a  (should be 1)\n", check_norm);

# Compact factored form
printf("\n  Factored form: P(r,0) = (6*r/R^2)*(1 - r/R)\n");
P_max_val := evalf(3 / (2 * R_val));
printf("  Maximum P_max = 3/(2R) = %.6e m^-1  at r = R/2\n", P_max_val);

# ------------------------------------------------------------
# PLOT 4 -- P(r,0) initial profile (dimensionless s=r/R)
# ------------------------------------------------------------

printf("\n=== PLOT 4: P(r,0) Initial Profile (dimensionless) ===\n");

# P(s,0) = 6*s*(1-s) / R  (normalised),  s in [0,1]
# For plotting use dimensionless P_tilde(s) = R * P(r,0) = 6*s*(1-s)

plt4 := plot(
    6 * s * (1 - s),
    s = 0 .. 1,
    color     = blue,
    thickness = 3,
    filled    = false,
    title     = "TauTimp: P(r,0)*R = 6s(1-s),  s=r/R  [normalised]",
    labels    = ["s = r/R", "P(r,0)*R"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes      = BOXED,
    gridlines = true
):
printf("  Plot 4 generated.\n");

# ------------------------------------------------------------
# PLOT 5 -- Probability Flux Density F(s) = P(s,0)/(4*Pi*s^2)
# (dimensionless: F_tilde = R^3*F = 6s(1-s)/(4*Pi*s^2) = 3(1-s)/(2*Pi*s))
# ------------------------------------------------------------

printf("\n=== PLOT 5: Flux Density F(s) = P(s,0)/(4*Pi*s^2) ===\n");

plt5 := plot(
    3 * (1 - s) / (2 * Pi * s),
```

```
    s = 0.02 .. 1,
    color     = red,
    thickness = 3,
    title     = "TauTimp: Flux F(s)*R^3 = 3(1-s)/(2*Pi*s)  [s=r/R]",
    labels    = ["s = r/R", "F(s)*R^3  [arb.]"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes      = BOXED,
    gridlines = true
):
printf("  Plot 5 generated.\n");

# ------------------------------------------------------------
# SECTION 11 -- Fourier-Sine Coefficients alpha_k
# ------------------------------------------------------------

printf("\n=== SECTION 11: Fourier-Sine Coefficients alpha_k ===\n");

# P(r,t) = sum_{k=1}^{inf} alpha_k * sin(k*Pi*s) * cos(k*Pi*tau)
# alpha_k = (2/R) * int_0^R P0(r)*sin(k*Pi*r/R) dr
#         = 2 * int_0^1 6*s*(1-s)*sin(k*Pi*s) ds
# For odd k: alpha_k = 48/(k^3*Pi^3*R)
# For even k: alpha_k = 0

N_coeff := 9:
alpha_k := Vector(N_coeff):
printf("  Fourier-Sine coefficients (odd k only):\n");
for k from 1 to N_coeff do
    if k mod 2 = 1 then
        alpha_k[k] := evalf(48 / (k^3 * Pi^3 * R_val));
        printf("    alpha_%d = %.6e m^-1\n", k, evalf(alpha_k[k]));
    else
        alpha_k[k] := 0.0;
        printf("    alpha_%d = 0  (even k)\n", k);
    end if;
end do:

# ------------------------------------------------------------
# SECTION 12 -- d'Alembert Evolution P(s,tau) -- N-mode sum
# ------------------------------------------------------------

printf("\n=== SECTION 12: d'Alembert Evolution P(s,tau) ===\n");

# P_tilde(s,tau) = R * P(r,t)

   #               = sum_{k odd, 1..N} (48/(k^3*Pi^3)) * sin(k*Pi*s)*cos
   (k*Pi*tau)

N_sum := 7:   # keep first 4 odd modes: k=1,3,5,7
```

```
P_tilde := 0:
for k from 1 to N_sum by 2 do
    P_tilde := P_tilde +
        (48 / (k^3 * Pi^3)) * sin(k * Pi * s) * cos(k * Pi * tau);
end do:

printf("  P_tilde(s,tau) [N=%d odd modes] built.\n", N_sum);

# ------------------------------------------------------------
# PLOT 6 -- P(s,tau) spacetime surface
# ------------------------------------------------------------

printf("\n=== PLOT 6: P(s,tau) Spacetime Surface ===\n");

plt6 := plot3d(
    P_tilde,
    s   = 0 .. 1,
    tau = 0 .. 2,
    grid      = [60, 60],
    style     = surface,
    shading   = zgrayscale,
    axes      = BOXED,
    orientation = [50, 65],
    title     = "TauTimp: P(s,tau)*R  [Fourier-sine, 4 odd modes]",
    labels    = ["s=r/R", "tau=ct/R", "P*R"],
    titlefont = [TIMES, BOLD, 12]
):
printf("  Plot 6 generated.\n");

# ------------------------------------------------------------
# PLOT 7 -- P(s,tau) snapshots at several tau values
# ------------------------------------------------------------

printf("\n=== PLOT 7: P(s,tau) Snapshots ===\n");

tau_vals   := [0, 1/4, 1/2, 3/4, 1]:
tau_colors := [blue, red, green, magenta, black]:
tau_labels := ["tau=0", "tau=1/4", "tau=1/2", "tau=3/4", "tau=1"]:

plots_snap := []:
for j from 1 to nops(tau_vals) do
    local tv, Psnap;
    tv    := evalf(tau_vals[j]);
    Psnap := 0:
    for k from 1 to N_sum by 2 do
        Psnap := Psnap +
            (48 / (k^3 * Pi^3)) * sin(k * Pi * s) * cos(k * Pi * tv);
    end do:
    p_snap := plot(
```

```
        Psnap,
        s = 0 .. 1,
        color     = tau_colors[j],
        thickness = 2,
        legend    = tau_labels[j]
    ):
    plots_snap := [op(plots_snap), p_snap];
end do:

plt7 := display(
    plots_snap,
    title
   = "TauTimp: P(s,tau)*R snapshots at tau = 0, 1/4, 1/2, 3/4, 1",
    labels    = ["s = r/R", "P(s,tau)*R"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes      = BOXED,
    gridlines = true
):
printf("  Plot 7 generated.\n");

# ------------------------------------------------------------
# PLOT 8 -- Airy function Ai(zeta) with zeros marked
# ------------------------------------------------------------

printf("\n=== PLOT 8: AiryAi(zeta) with eigenvalue zeros ===\n");

Ai_plot := plot(
    AiryAi(zeta),
    zeta = -10 .. 3,
    color     = blue,
    thickness = 2,
    title     = "Airy function Ai(zeta): zeros give TauTimp eigenvalues",
    labels    = ["zeta", "Ai(zeta)"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes      = BOXED,
    gridlines = true
):

# Mark zeros with vertical dashed lines
zero_lines := []:
for n from 1 to 4 do
    local zn;
    zn := evalf(AiryZeros[n]);
    vl := plot(
        [[zn, -0.5], [zn, 0.5]],
        color  = red,
        linestyle = dash,
```

```
        thickness = 1
    ):
    zero_lines := [op(zero_lines), vl];
end do:

# zero line y=0
hline := plot([[-10, 0], [3, 0]], color=black, thickness=1):

plt8 := display(
    [Ai_plot, op(zero_lines), hline],
    title
   = "Airy function Ai(zeta): zeros a_n give TauTimp eigenvalues",
    labels    = ["zeta", "Ai(zeta)"],
    labelfont = [TIMES, BOLD, 12],
    titlefont = [TIMES, BOLD, 13],
    axes      = BOXED,
    gridlines = true
):
printf("  Plot 8 generated.\n");

# ------------------------------------------------------------
# SECTION 13 -- Numerical Verification of Normalization
# ------------------------------------------------------------

printf("\n=== SECTION 13: Numerical Verification ===\n");

# int_0^1  6*s*(1-s) ds  should equal 1
norm_check := evalf(int(6*s*(1-s), s=0..1));
printf("  int_0^1 P_tilde(s,0) ds = %.10f  (should be 1.0)\n",
    norm_check);

# Verify BC
BC_check := evalf(subs(s=1, 6*s*(1-s)));
printf("  P_tilde(1,0) = %.10f  (should be 0.0)\n", BC_check);
BC_check0 := evalf(subs(s=0, 6*s*(1-s)));
printf("  P_tilde(0,0) = %.10f  (should be 0.0)\n", BC_check0);

# Peak location and value
s_peak := 0.5;
P_peak := evalf(subs(s=s_peak, 6*s*(1-s)));
printf("  Peak: P_tilde(1/2,0) = %.10f  at s=0.5  (should be 1.5)\n",
    P_peak);

# Fourier series convergence check at tau=0, s=0.5
P_fourier_check := 0:
for k from 1 to 99 by 2 do
    P_fourier_check := P_fourier_check +
        evalf(48 / (k^3 * Pi^3)) * sin(k * Pi * 0.5);
end do:
```

```
  printf("  Fourier sum at s=0.5, tau=0 (50 terms) = %.8f  (expect 1.5)\n",
         evalf(P_fourier_check));

  # Eigenfrequency table
  printf("\n  === Eigenfrequency Table ===\n");
  printf("  %4s  %12s  %14s  %14s\n", "n", "|a_n|", "lambda_n [rad/m]",
      "omega_n [rad/s]");
  for n from 1 to N_modes do
      printf("  %4d  %12.6f  %14.6e  %14.6e\n",
             n,
             abs(evalf(AiryZeros[n])),
             evalf(lambda_vals[n]),
             evalf(omega_vals[n]));
  end do:

  # ------------------------------------------------------------
  # SECTION 14 -- Display All Plots
  # ------------------------------------------------------------

  printf("\n=== SECTION 14: Displaying All Plots ===\n");

  display(plt1);   # Radial eigenfunctions
  display(plt2);   # f_1(s,tau) spacetime surface
  display(plt3);   # H_1 polar cross-section
  display(plt4);   # P(r,0) initial profile
  display(plt5);   # Flux density F(s)
  display(plt6);   # P(s,tau) spacetime surface
  display(plt7);   # P(s,tau) snapshots
  display(plt8);   # Airy function with zeros

  printf("\n=== ALL SECTIONS COMPLETE ===\n");
  printf("TauTimp SphericalWave -- Viorel Popescu, April 2026\n");

  # ============================================================
  # END OF TRANSCRIPT
  # ============================================================

=== SECTION 1: Physical Constants ===
  R  = 3.922800e+22 m
  c  = 2.997925e+08 m/s

=== SECTION 2: TauTimp Wave Equation ===

Error, recursive assignment

                                PDE_TauTimp := 0 = 0

  TauTimp PDE (spherical symmetry):

                                      0 = 0

=== SECTION 3: Separation of Variables ===
```

$$u_sol := C1\cos(\omega t) + C2\sin(\omega t)$$

Temporal solution u(t):

$$C1\cos(\omega t) + C2\sin(\omega t)$$

Error, recursive assignment

$$RadialODE := \frac{\lambda^2 r g}{R} = 0$$

Radial ODE:

$$\frac{\lambda^2 r g}{R} = 0$$

=== SECTION 4: Airy Equation Reduction ===

$$AiryEq := \frac{d^2}{d\zeta^2} v(\zeta) - \zeta v(\zeta) = 0$$

Standard Airy equation:

$$\frac{d^2}{d\zeta^2} v(\zeta) - \zeta v(\zeta) = 0$$

$$v_general := D1\,\mathrm{AiryAi}(\zeta) + D2\,\mathrm{AiryBi}(\zeta)$$

General solution v(zeta):

$$D1\,\mathrm{AiryAi}(\zeta) + D2\,\mathrm{AiryBi}(\zeta)$$

Physical solution (D2=0, regularity at r->0):

$$v_phys := D1\,\mathrm{AiryAi}(\zeta)$$

$$D1\,\mathrm{AiryAi}(\zeta)$$

=== SECTION 5: Eigenvalue Quantization ===

Airy zeros a_n (zeros of Ai):

a_1 = -2.338107
a_2 = -4.087949
a_3 = -5.520560
a_4 = -6.786708
a_5 = -7.944134

Eigenvalues lambda_n and omega_n:

n=1: lambda_1 = 9.113822e-23 rad/m, omega_1 = 2.732255e-14 rad/s
n=2: lambda_2 = 2.106988e-22 rad/m, omega_2 = 6.316592e-14 rad/s
n=3: lambda_3 = 3.306576e-22 rad/m, omega_3 = 9.912866e-14 rad/s
n=4: lambda_4 = 4.507052e-22 rad/m, omega_4 = 1.351180e-13 rad/s
n=5: lambda_5 = 5.707864e-22 rad/m, omega_5 = 1.711175e-13 rad/s

=== SECTION 6: Radial Eigenfunctions g_n(r) ===

$$g_n := (n_idx, rr) \mapsto \frac{\mathrm{AiryAi}\left(\frac{\left|AiryZeros_{n_idx}\right| \cdot rr}{R_val}\right)}{rr}$$

$$g_dim := (n_idx, s) \mapsto \frac{\mathrm{AiryAi}\left(\left|evalf\left(AiryZeros_{n_idx}\right)\right| \cdot s\right)}{s}$$

```
  Radial eigenfunction (dimensionless, s=r/R):
    g_n(s) = AiryAi(|a_n|*s) / s

=== PLOT 1: Radial Eigenfunctions g_n(s) ===

Warning, embedded local declaration of name `an_abs` ignored outside
procedure or module
Warning, embedded local declaration of name `an_abs` ignored outside
procedure or module
Warning, embedded local declaration of name `an_abs` ignored outside
procedure or module
Warning, embedded local declaration of name `an_abs` ignored outside
procedure or module
  Plot 1 generated.

=== SECTION 7: Full Wave Field f(r,t) -- Mode 1 ===
```

$$a1_abs := 2.338107410$$

$$f1_expr := (s, \tau) \mapsto \frac{\mathrm{AiryAi}(a1_abs \cdot s) \cdot \cos\left(a1_abs^{3/2} \cdot \tau\right)}{s}$$

```
  f_1(s,tau) = Ai(|a1|*s)/s * cos(|a1|^(3/2)*tau)
  |a1| = 2.338107

=== PLOT 2: Spacetime Surface f_1(s,tau) ===
  Plot 2 generated.

=== SECTION 8: H(r,theta,t) -- Angular Field ===
```

$$H1_expr := (s, th, \tau) \mapsto \frac{\mathrm{AiryAi}(a1_abs \cdot s) \cdot \cos\left(a1_abs^{3/2} \cdot \tau\right) \cdot \cos(th)}{s}$$

```
  H_1(s,theta,tau) = f_1(s,tau)*cos(theta)  [ell=1 mode]

=== PLOT 3: Polar map of H_1(s,theta) at tau=0 ===
  Plot 3 generated.

=== SECTION 9: Probability Density PDE ===
```

$$PDE_P := \frac{\partial^2}{\partial t^2} P(r, t) - c^2 \left(\frac{\partial^2}{\partial r^2} P(r, t)\right) = 0$$

```
  Probability PDE:
```

$$\frac{\partial^2}{\partial t^2} P(r,t) - c^2 \left(\frac{\partial^2}{\partial r^2} P(r,t) \right) = 0$$

```
=== SECTION 10: Degree-2 Initial Profile P(r,0) ===
```

$$A_sym := -\frac{6}{R^3}$$

$$B_sym := \frac{6}{R^2}$$

$$C_sym := 0$$

```
  Symbolic coefficients:
    A = -6/R^3
    B =  6/R^2
    C =  0
```

$$A_num := -9.939459090 \times 10^{-68}$$

$$B_num := 3.899051012 \times 10^{-45}$$

$$C_num := 0.$$

```
  Numerical values (R = 3.9228e+22 m):
    A = -9.939459e-68 m^-3
    B = 3.899051e-45 m^-2
    C = 0
```

$$P0_sym := -\frac{6\,r^2}{R^3} + \frac{6\,r}{R^2}$$

```
  P(r,0) symbolic form:
```

$$-\frac{6\,r^2}{R^3} + \frac{6\,r}{R^2}$$

$$check_BC := 0$$

$$check_norm := 1$$

```
  Verification P(R,0) = 0   (should be 0)
  Verification integral = 1   (should be 1)

  Factored form: P(r,0) = (6*r/R^2)*(1 - r/R)
```

$$P_max_val := 3.823799326 \times 10^{-23}$$

```
  Maximum P_max = 3/(2R) = 3.823799e-23 m^-1   at r = R/2

=== PLOT 4: P(r,0) Initial Profile (dimensionless) ===
  Plot 4 generated.

=== PLOT 5: Flux Density F(s) = P(s,0)/(4*Pi*s^2) ===
```

```
  Plot 5 generated.

=== SECTION 11: Fourier-Sine Coefficients alpha_k ===
  Fourier-Sine coefficients (odd k only):
    alpha_1 = 3.946349e-23 m^-1
    alpha_2 = 0  (even k)
    alpha_3 = 1.461611e-24 m^-1
    alpha_4 = 0  (even k)
    alpha_5 = 3.157079e-25 m^-1
    alpha_6 = 0  (even k)
    alpha_7 = 1.150539e-25 m^-1
    alpha_8 = 0  (even k)
    alpha_9 = 5.413373e-26 m^-1

=== SECTION 12: d'Alembert Evolution P(s,tau) ===
  P_tilde(s,tau) [N=7 odd modes] built.

=== PLOT 6: P(s,tau) Spacetime Surface ===
  Plot 6 generated.

=== PLOT 7: P(s,tau) Snapshots ===

Warning, embedded local declaration of name `tv` ignored outside procedure or
module
Warning, embedded local declaration of name `Psnap` ignored outside procedure
or module
Warning, embedded local declaration of name `tv` ignored outside procedure or
module
Warning, embedded local declaration of name `Psnap` ignored outside procedure
or module
Warning, embedded local declaration of name `tv` ignored outside procedure or
module
Warning, embedded local declaration of name `Psnap` ignored outside procedure
or module
Warning, embedded local declaration of name `tv` ignored outside procedure or
module
Warning, embedded local declaration of name `Psnap` ignored outside procedure
or module
Warning, embedded local declaration of name `tv` ignored outside procedure or
module
Warning, embedded local declaration of name `Psnap` ignored outside procedure
or module
  Plot 7 generated.

=== PLOT 8: AiryAi(zeta) with eigenvalue zeros ===

Warning, embedded local declaration of name `zn` ignored outside procedure or
module
Warning, embedded local declaration of name `zn` ignored outside procedure or
```

```
module
Warning, embedded local declaration of name `zn` ignored outside procedure or
module
Warning, embedded local declaration of name `zn` ignored outside procedure or
module
  Plot 8 generated.

=== SECTION 13: Numerical Verification ===
```

$$norm_check := 1.$$

```
  int_0^1 P_tilde(s,0) ds = 1.0000000000  (should be 1.0)
```

$$BC_check := 0.$$

```
  P_tilde(1,0) = 0.0000000000  (should be 0.0)
```

$$BC_check0 := 0.$$

```
  P_tilde(0,0) = 0.0000000000  (should be 0.0)
```

$$s_peak := 0.5$$

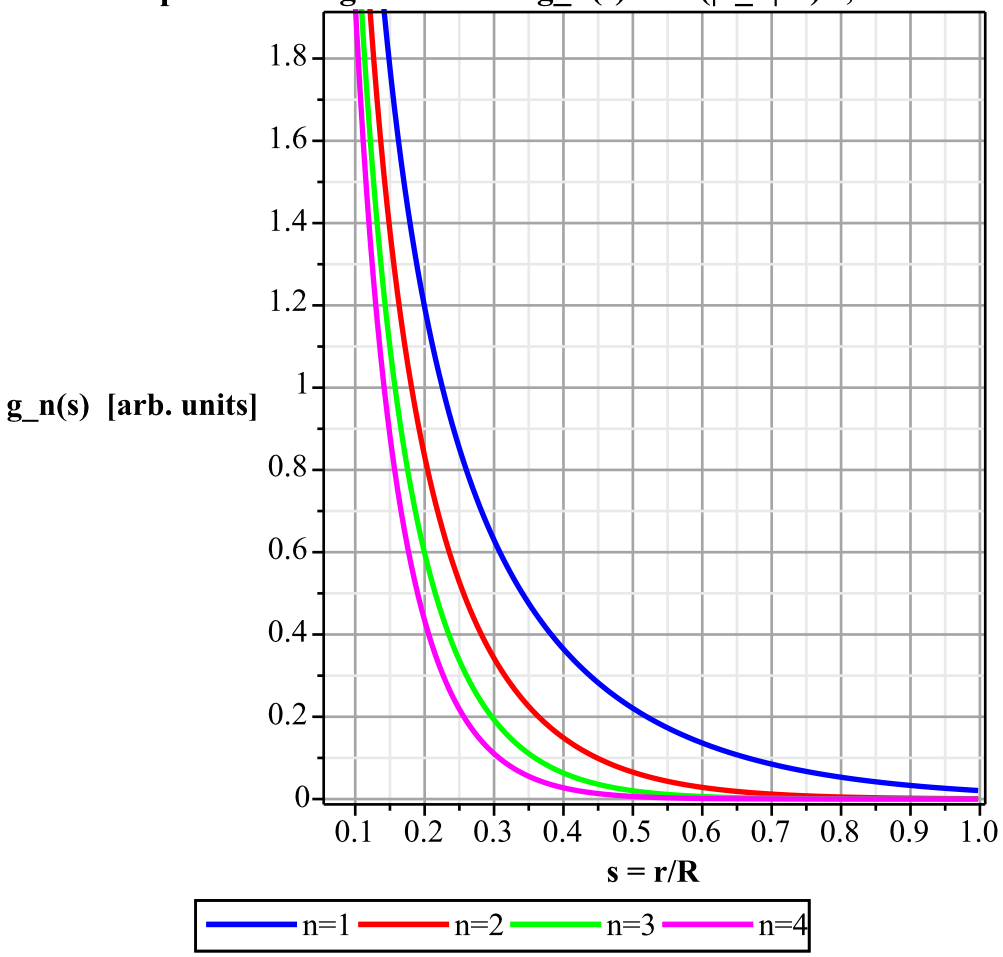
$$P_peak := 1.50$$

```
  Peak: P_tilde(1/2,0) = 1.5000000000  at s=0.5  (should be 1.5)
  Fourier sum at s=0.5, tau=0 (50 terms) = 1.49999922  (expect 1.5)

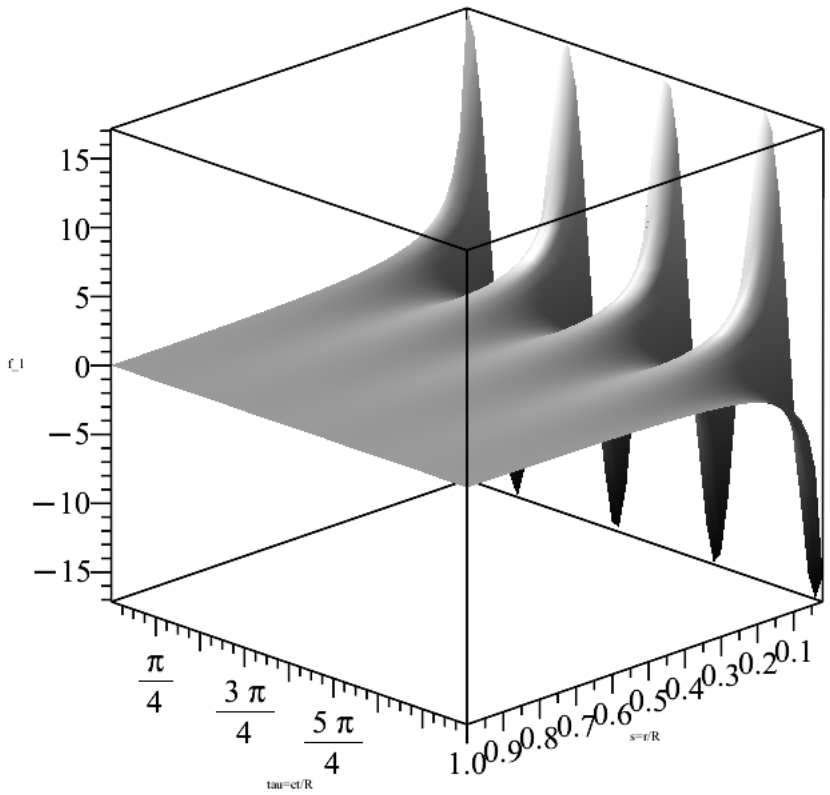
  === Eigenfrequency Table ===
     n          |a_n|  lambda_n [rad/m]  omega_n [rad/s]
     1       2.338107    9.113822e-23    2.732255e-14
     2       4.087949    2.106988e-22    6.316592e-14
     3       5.520560    3.306576e-22    9.912866e-14
     4       6.786708    4.507052e-22    1.351180e-13
     5       7.944134    5.707864e-22    1.711175e-13

=== SECTION 14: Displaying All Plots ===
```

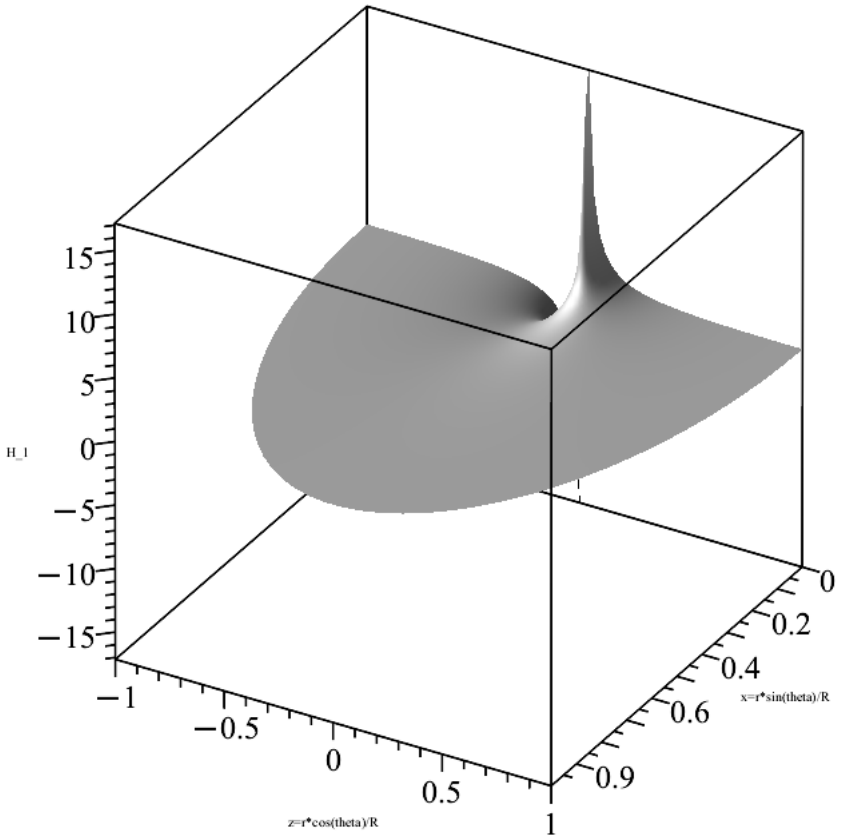
TauTimp: Radial Eigenfunctions g_n(s) = Ai(|a_n|*s)/s, s = r/R



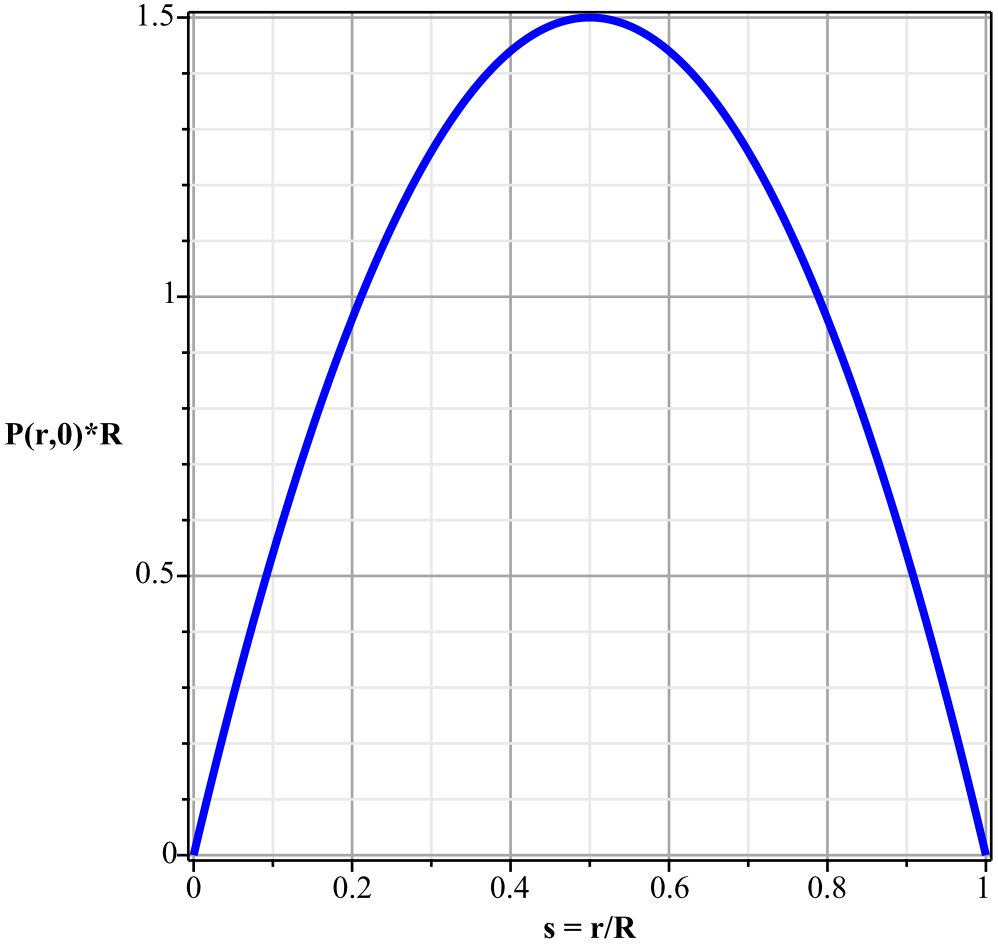
TauTimp: f_1(s,tau) = Ai(|a1|*s)/s * cos(|a1|^(3/2)*tau)



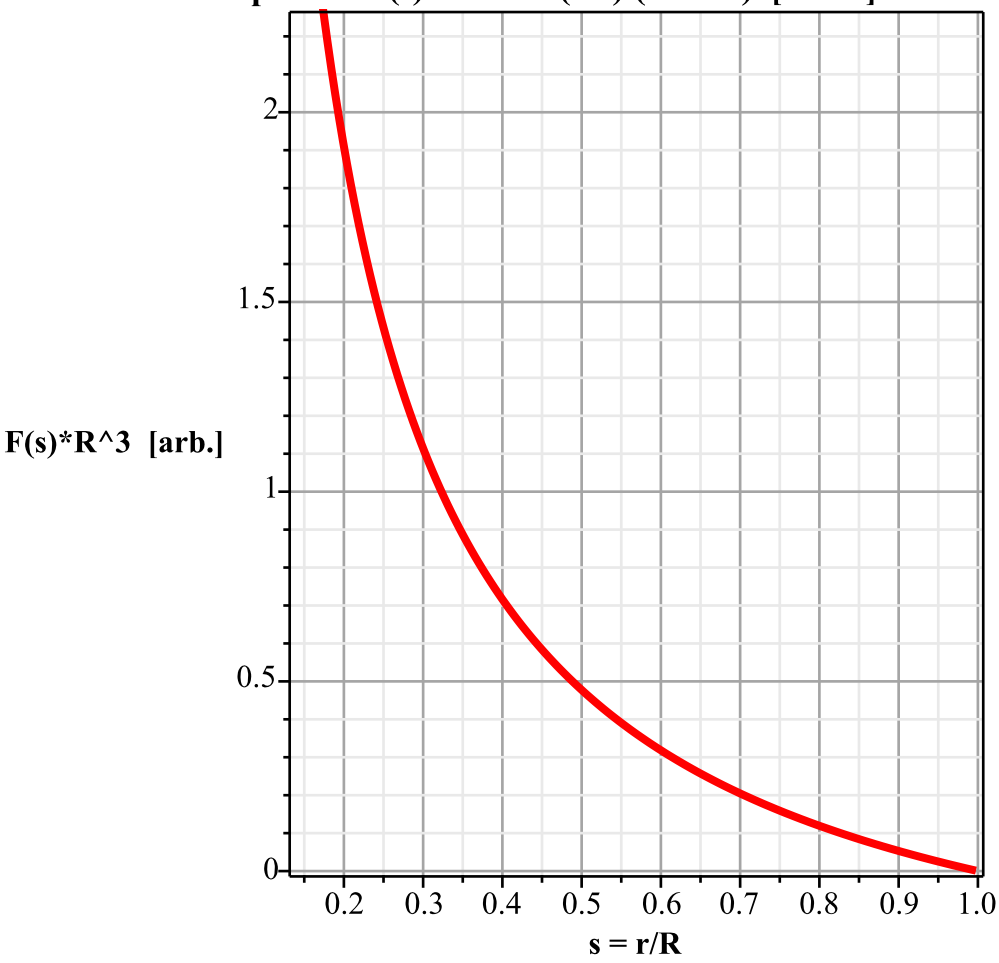
TauTimp: H_1(r,theta,0) polar cross-section (ell=1, tau=0)



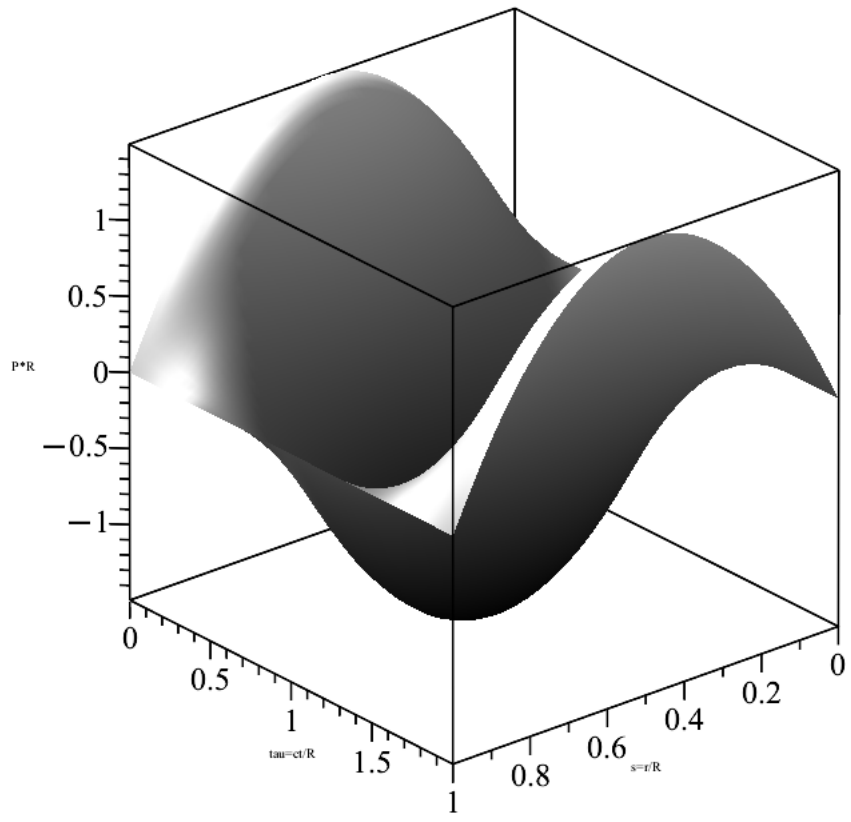
TauTimp: P(r,0)*R = 6s(1-s), s=r/R [normalised]

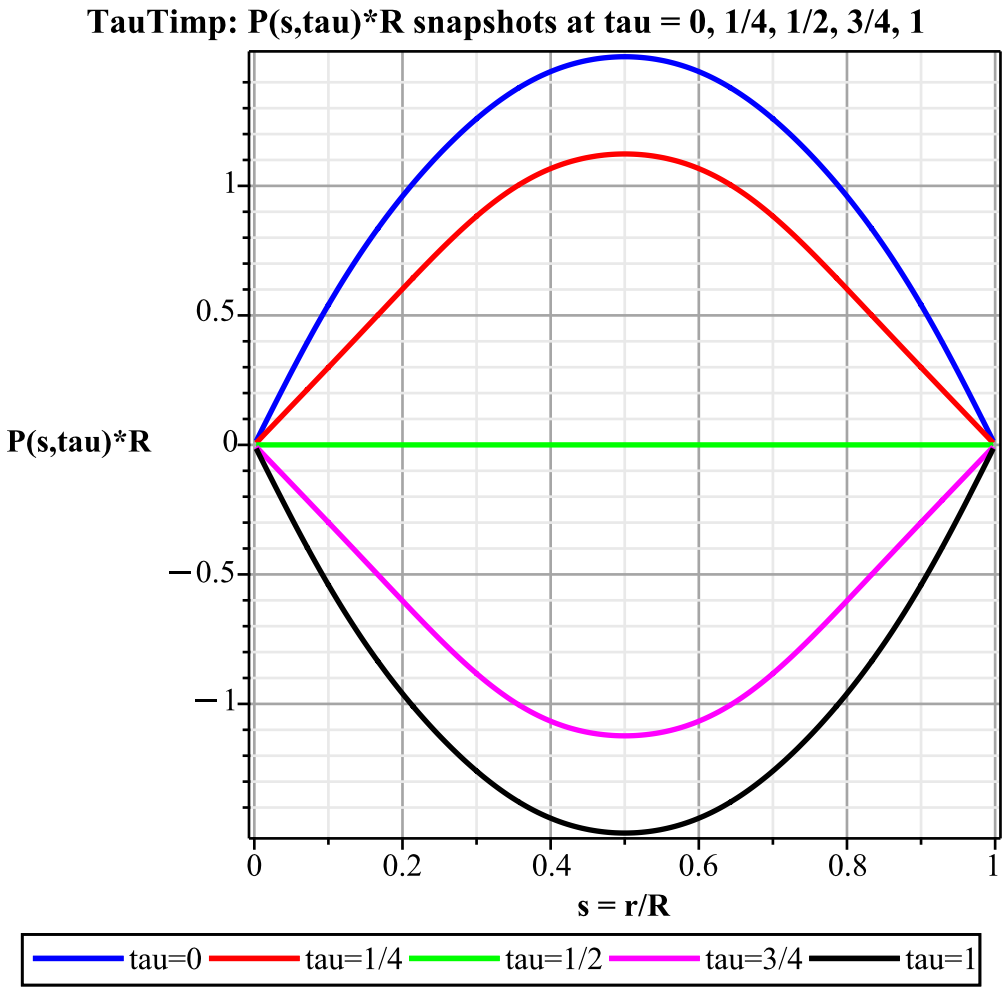


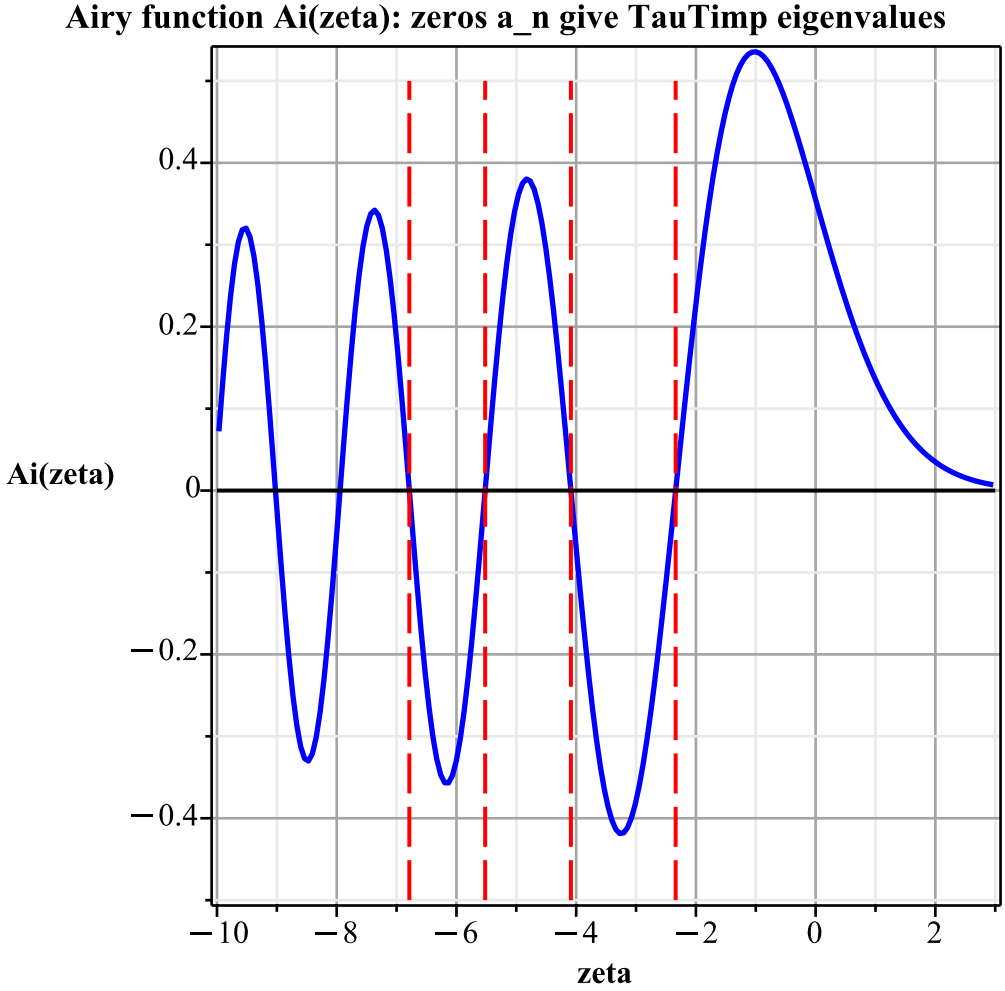
TauTimp: Flux F(s)*R^3 = 3(1-s)/(2*Pi*s) [s=r/R]



TauTimp: P(s,tau)*R [Fourier-sine, 4 odd modes]








```
=== ALL SECTIONS COMPLETE ===
```

```
TauTimp SphericalWave -- Viorel Popescu, April 2026

>
```